Ascend: Functional Specification

Project	Ascend: AI-powered growth intelligence for small businesses
Document	Functional Specification
Course	P6 Special Topics
Team	Aaryaman Chopra, Ethan de Guzman, Palash Desarda, Kaiyaan Faisal
Repository	github.com/EDGDevelopment/Ascend (documents are in the docs folder)
Version	Draft 1.0, November 6, 2026

1. Purpose and Scope

Ascend is a web app that turns a small business's or non-profit's monthly numbers into a 12-month forecast with a likely range, then answers what an owner or board asks next: what the organisation is worth or how long its cash lasts, and what happens if prices, volume or costs move. This draft specifies what the system must do, marks what is built as of Sprint Review 2 (October 27, 2026), and lists what remains for November to January.

In scope

- Account sign-up, sign-in and password recovery.
- Upload of 12 to 120 months of revenue and expenses (Excel or CSV) and the details the model needs.
- 12-month forecast with a range; valuation (local business) or cash runway (non-profit); and a toolkit of What if, Break-even, Cash safety, Alerts and Customers.
- A data manager for keeping several uploaded files.
- Overview, Forecasts, Opportunities, Data and Model dashboard pages.

Out of scope for this course deliverable

- Live connections to accounting, point-of-sale, payroll or inventory systems (the pitch's operational and external data feeds).
- Industry-specific models and a cross-business learning network (the pitch's later scale stages).
- Native mobile apps. The web app is responsive down to phone width.

2. Users and Use Cases

There are two kinds of client, chosen when a file is uploaded. Both are people without a data team, which is the gap the pitch describes.

User	Who they are	What they need from Ascend
Local business owner	Runs a cafe, shop, salon or similar business with 12 to 120 months of sales and costs.	A forecast with a range, an estimate of what the business is worth, and a way to test price, volume and cost changes.

User	Who they are	What they need from Ascend
Non-profit board or director	Oversees a charity or program funded by grants and giving.	Months of cash on hand, chance of running short, funding concentration and the effect of grants still to land.

Primary use cases

1. Sign up, sign in, and recover a forgotten password.
2. Upload a monthly spreadsheet (or enter months in the form) and choose business or non-profit.
3. Read the next 12 months with a likely range, and see how accurate the model was on the client's own recent months.
4. Try a what-if change (price, volume, costs) and see the effect on profit and value.
5. See break-even, cash safety, unusual-month alerts and customer economics.
6. Keep several spreadsheets: switch between them, rename, download or delete.
7. See ranked growth levers on the Opportunities page.
8. Switch between light and dark themes on any screen size.

3. System Overview

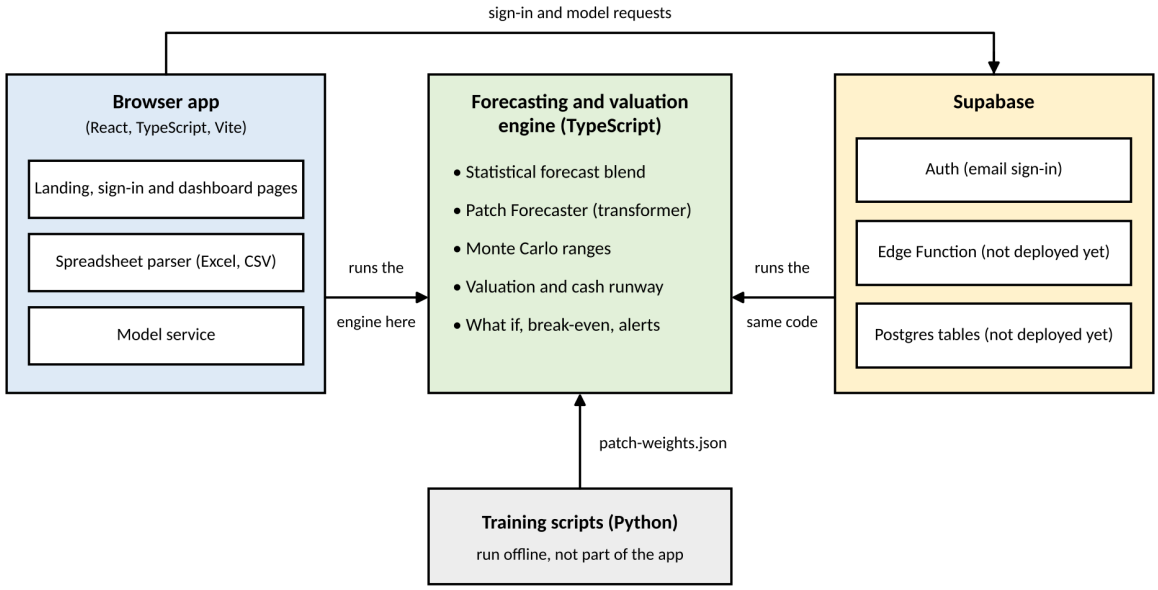


Figure 1. System architecture. The Edge Function and database tables are written but not deployed yet.

The browser app parses a spreadsheet, then either calls the Edge Function or runs the same engine files itself. Supabase handles sign-in and stores each client's data under row-level security. The Patch Forecaster is trained offline in Python and shipped as a 2.4 MB weights file. The app is built with React 19, TypeScript, Vite, Tailwind CSS v4 and Recharts.

4. Functional Requirements

Twenty-two of the 24 requirements are built and two are partial: the backend is written but not yet deployed to a live Supabase project. Status is as of Sprint Review 2 (October 27, 2026).

ID	Requirement	Status	Where it lives
FR-1	A visitor can sign up, sign in and sign out with email and password on Supabase Auth. Without Supabase credentials the app runs in demo mode.	Built	src/auth/
FR-2	A user can reset a forgotten password by emailed link (/forgot-password, /reset-password).	Built	src/pages/auth/
FR-3	All /dashboard routes require sign-in; signed-out visitors are redirected.	Built	src/auth/ProtectedRoute.tsx
FR-4	A user can upload an .xlsx or .csv: a Monthly sheet (Month, Revenue, Expenses) plus a details sheet (Business, or Organisation and Pipeline). Templates can be downloaded.	Built	src/valuation/spreadsheet.ts
FR-5	The parser accepts months as 2025-03, Mar 2025, 3/2025, 3/1/2025 or an Excel date; money with commas or currency symbols; rates as 0.08, 8 or 8%.	Built	src/valuation/spreadsheet.ts
FR-6	The system rejects files with fewer than 12 or more than 120 months, a missing month, negative revenue or partial expenses, and says why in plain words.	Built	model.ts; tests/model.test.mjs
FR-7	Expenses are optional for a local business and required for a non-profit.	Built	model.ts
FR-8	The system produces a 12-month forecast of revenue, expenses and cash flow, with a range from 3,000 Monte Carlo simulated futures.	Built	forecast.ts
FR-9	The forecasting method depends on history: 12 to 23 months uses last year scaled by recent growth; 24 or more blends Holt-Winters, last-year-scaled and Theta; 36 or more adds the Patch Forecaster at half weight.	Built	forecast.ts; patch.ts
FR-10	Each run hides the latest months, forecasts them and reports the error against a same-month-last-year guess (the in-app accuracy self-test).	Built	forecast.ts
FR-11	For a given seed the browser and the Edge Function return identical numbers.	Built	rand.ts; tests
FR-12	For a local business the system reports a value range: earnings multiple 40%, cash flow 30%, excess earnings 30% (55/45 without equipment), floored at saleable equipment and stock.	Built	local.ts; dcf.ts
FR-13	For a non-profit the system reports months of cash, operating reserve ratio, chance of running dry in 12 months, chance of surplus, funding concentration and the effect of grants in the pipeline.	Built	nonprofit.ts

ID	Requirement	Status	Where it lives
FR-14	What if: price, volume and cost sliders, with a ranking of the biggest levers.	Built	scenario.ts; WhatIf.tsx
FR-15	Break-even: fixed and variable costs fitted from the client's own months.	Built	costs.ts; BreakEven.tsx
FR-16	Cash safety: how long cash lasts and the chance of running short.	Built	cash.ts; CashSafety.tsx
FR-17	Alerts: unusual months are flagged.	Built	signals.ts; Alerts.tsx
FR-18	Customers: lifetime value against the cost to win a customer.	Built	scenario.ts; Customers.tsx
FR-19	A user can keep many uploaded files, switch between them, rename, download or delete any.	Built	DataSourcesPage.tsx
FR-20	Overview, Forecasts, Opportunities, Data and Model pages show only the client's own data; with no upload they prompt for one. Opportunities ranks levers by profit and value effect.	Built	src/dashboard/pages/
FR-21	The theme can switch between light and dark, and screens work down to phone width.	Built	src/theme/
FR-22	The model runs in a Supabase Edge Function that requires a signed-in user, allows only listed origins and limits each user to 10 runs a minute; if it is unreachable the browser runs the same engine.	Partial: written, not deployed	supabase/functions/valuation
FR-23	Each uploaded file and its latest result are stored per user in valuation_datasets with row-level security; without Supabase they stay in browser storage.	Partial: migration written, not deployed	supabase/migrations/0003
FR-24	Profiles and businesses tables hold per-user rows that only the owner can read or change.	Built	supabase/migrations/0001

5. Data Requirements

The client supplies one row per month; everything else is derived. Each client's data is visible only to that client.

Item	Rule
History length	12 to 120 consecutive months. 24 or more is recommended; 36 or more unlocks the Patch Forecaster.
Required columns	Month and Revenue. Expenses are optional for a local business and required for a non-profit.
Details sheet	Local business: a Business sheet of inputs the valuation needs. Non-profit: an Organisation sheet plus a Pipeline sheet of grants and campaigns in progress.
Rejected input	Fewer than 12 months, a gap in months, negative revenue, partly filled expenses, simulation settings out of range.

Item	Rule
Test files	14 synthetic files in data/sample-data cover seasonality, growth, decline, a COVID-style dip, alternative date and currency formats, and three files that must be rejected.

Stored data (Supabase, row-level security on every table)

Table	Holds	Migration
profiles	One row per user: full name and business name, filled by a trigger at sign-up.	0001
businesses	A user's businesses: name, industry, location.	0001
valuation_datasets	One row per uploaded spreadsheet: inputs, latest output, timestamps.	0003
valuation_models	An earlier one-row-per-user design, left in place and unused.	0002

The service-role key is never used in the repo or the browser; only the publishable key is. Real credentials live in .env.local, which is git-ignored. The data-access and privacy review is scheduled for January (see the Project Schedule).

6. Non-Functional Requirements

Forecast accuracy is the requirement that matters most, and it has been measured on public national data only, not yet on real small-business data.

Area	Requirement	Evidence today
Accuracy	At least 94% monthly accuracy (100% minus average monthly percentage error) with 36 months of history.	95.2% statistical engine, 95.5% with the Patch Forecaster, on 31 real monthly series.
Range honesty	The stated range should contain the actual value about 8 times in 10.	85% at 36 months and 82% at 18 months; as low as 60 to 65% on some short series.
Honest claims	The app shows the client's own accuracy from their recent months and never promises the national-data figure.	In-app self-test badge (FR-10).
Security	Only signed-in users reach the model; each user sees only their own rows; no secret keys in the browser or repo.	JWT on the Edge Function, row-level security on every table, publishable key only, .env.local git-ignored.
Abuse control	Origins are restricted and each user is limited to 10 runs a minute.	Built in the Edge Function; the limit is best effort because it is held in memory per instance.
Portability	One forecasting engine runs unchanged in the browser and the Edge Function.	No dependencies and no browser, Node or Deno APIs inside the engine folder.
Reliability	If the backend is unreachable the app still produces a forecast.	Browser fallback in service.ts.
Usability	Guided three-step flow, plain-language errors, light and dark themes, phone-width layout.	Financial model page; validation messages.

Area	Requirement	Evidence today
Maintainability	Type-checked TypeScript, linted, with automated tests.	npm run typecheck, npm run lint, npm test.
Performance	Forecast and simulation finish without freezing the page.	Not yet timed; to be measured in November.

7. Constraints, Assumptions and Known Gaps

The largest gap is that the backend has never run against a live Supabase project and the model has never seen real small-business data.

Constraints and assumptions

- Course project: a four-person team, finishing with a demo in late January.
- The forecast assumes the future behaves like the past. A new competitor, a lost contract or a move is not in the numbers.
- Valuation methods cannot be validated against sale prices, because no public data of small-business sales exists. The preset multiples (about 1.5x to 4x) are rules of thumb.
- The Patch Forecaster is a small transformer (about 450,000 parameters) trained on public M4 and US retail series. It adds roughly 0.2 points over the statistical blend.

Known gaps

Gap	Effect	Planned response
Edge Function and migration 0003 not deployed	The app falls back to the browser; results are not saved server-side.	Deploy to a test project and run end to end (Ethan).
No real small-business data	Accuracy is proven only on national retail totals, which are smoother than one shop.	Collect and clean real data in November (Kaiyaan); retest and tune (Aaryaman).
Short history is weaker	With 18 months, ranges held the truth only 60 to 65% of the time on some series.	Ask for 24 or more months; widen ranges for short history.
Engine of about 5,000 lines, one author	Risk of an unnoticed maths error.	A teammate reviews it against hand calculations (Palash).
Edge Function imports weights with a JSON import attribute	Needs a recent Deno; may fail on deploy.	Switch to reading the file at start-up if it fails.
README status section says dashboard data is mocked	Out of date since Sprint 2, when sample data was removed.	Refresh the README.
Pitch features not built: inventory, customer and external-data signals	The product covers financial data only.	Out of scope; stated in section 1.

8. Acceptance Criteria and Test Plan

The project is accepted when each row below passes. The sample files in data/sample-data are the shared test inputs.

#	Check	How	Pass when	Status
1	Bad files are refused	Upload the three EDGE-* files	Each is rejected with a plain reason: under 12 months, a missing month, negative revenue	Automated tests cover the rules
2	Formats are accepted	Upload the month-name, US-date and dollar-sign files	The same months and amounts are read as from cafe-36-months	Manual
3	Forecast runs end to end	Upload cafe-36-months	12-month forecast and range appear; the accuracy badge shows	Driven in Chrome
4	Alerts fire on a shock	Upload restaurant-covid-style-dip	The collapse months are flagged; normal months are not	Backtest: 22 of 26 retail categories flagged
5	Non-profit flow	Upload nonprofit-36-months as Non-profit	Runway, reserve ratio and funding mix appear	Driven in Chrome
6	Repeatable results	Run the same file and seed twice, in browser and on the Edge Function	Identical numbers	Browser tested; Edge Function pending
7	Accuracy holds	node scripts/backtest.mjs on the 31-series folder	Monthly accuracy of 94% or more at 36 months of history	95.2% measured
8	Real-data accuracy	Score the model on collected small-business data	Reported honestly, with ranges re-tuned if they miss	Planned, November to December
9	Backend deployed	supabase db push and functions deploy; sign in and upload	Result saved to valuation_datasets and reloads; another user cannot read it	Planned, November
10	Access control	Two accounts, each opens the other's data	Refused by row-level security	Planned, January
11	Code health	npm run lint, npm run typecheck, npm test	All pass	npm test: 8 of 8 pass

The test suite is one file of 8 tests focused on validation and determinism. The valuation, cash and scenario maths were checked against a hand calculation and known-truth cases but have no automated tests yet; adding them is a November work package.